Documento de Arquitectura — Sistema SmartPlantBase (Fase 2)

Proyecto Matera Inteligente · Arquitectura Hexagonal (Puertos y Adaptadores) · **Estudiante:** Esteban Castro · **Semestre:** 2026-2 · **Tecnologías:** Python 3.14, Flask, SQLite, PyJWT, Werkzeug, pytest

1. Diagrama Hexagonal del Sistema

Para la segunda fase de SmartPlantBase estructuramos la aplicación mediante el patrón de **Puertos y Adaptadores (Alistair Cockburn, 2005)**. El objetivo principal es aislar la lógica botánica y las reglas de negocio de cualquier detalle tecnológico: protocolos de transporte, frameworks web, controladores de persistencia o mecanismos criptográficos.

La Figura 1 ilustra la vista de referencia de nuestra arquitectura, reflejando con exactitud los paquetes, casos de uso atómicos, puertos segregados y adaptadores implementados en el código:

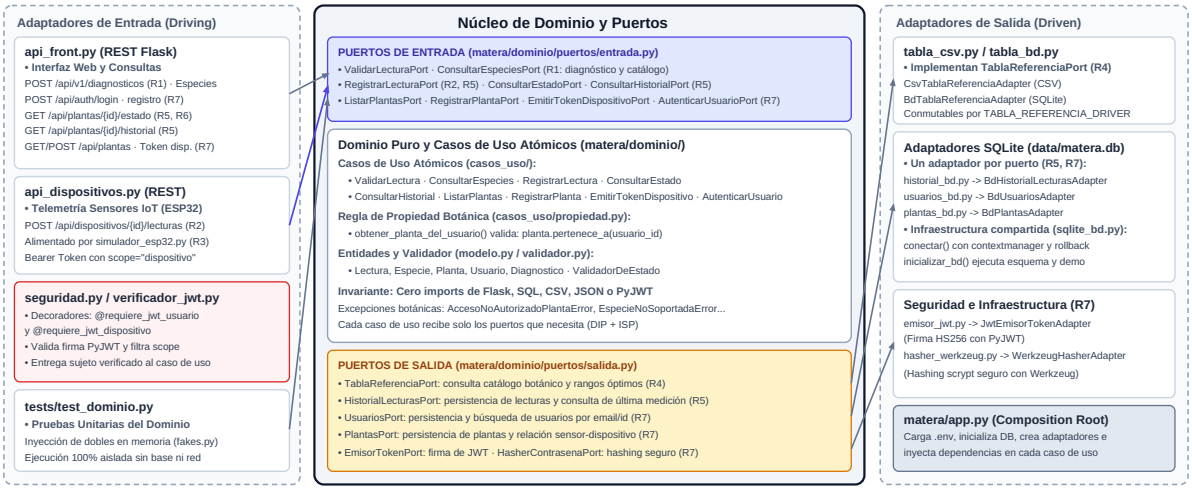


Figura 1. Vista hexagonal del sistema con adaptadores, puertos segregados, casos de uso atómicos y el Composition Root.

El flujo de dependencias apunta siempre hacia adentro:

- **Adaptadores de entrada (primarios):** Traducen estímulos del exterior a invocaciones de los puertos de entrada. El módulo `api_front.py` atiende solicitudes web del usuario y `api_dispositivos.py` recibe la telemetría periódica del ESP32. El filtro `seguridad.py` valida tokens y entrega identidades limpias.
- **Núcleo de dominio:** Agrupa casos de uso atómicos (`matera/dominio/casos_uso/`), entidades botánicas (`Lectura`, `Especie`, `Planta`, `Usuario`, `Diagnostico`) y evaluadores de reglas (`ValidadorDeEstado`). No contiene librerías externas ni referencias a HTTP o SQL.
- **Adaptadores de salida (secundarios):** Implementan los puertos de salida para interactuar con la infraestructura: almacenamiento especializado en SQLite (`historial_bd.py`, `usuarios_bd.py`, `plantas_bd.py`), catálogo botánico (`tabla_csv.py`, `tabla_bd.py`), firma de tokens (`emisor_jwt.py`) y hash criptográfico (`hasher_werkzeug.py`).

2. Tabla de Puertos y Adaptadores

Los puertos desacoplan el núcleo mediante contratos explícitos en `matera/dominio/puertos/` (`entrada.py` y `salida.py`). Cada caso de uso atómico implementa un puerto de entrada, y cada adaptador satisface un puerto de salida:

Puerto (Contrato)	Tipo	Adaptador Concreto / Implementación	Responsabilidad en el Negocio
ValidarLecturaPort	Entrada	`ValidarLectura` *(invocado por `api_front`)*	Evalúa lecturas manuales contra los rangos de la especie (R1).
ConsultarEspeciesPort	Entrada	`ConsultarEspecies` *(invocado por `api_front`)*	Provee catálogo de especies para selectores de la interfaz web (R1).
RegistrarLecturaPort	Entrada	`RegistrarLectura` *(invocado por `api_dispositivos`)*	Procesa telemetría ESP32, evalúa estado botánico y persiste (R2, R5).
ConsultarEstadoPort	Entrada	`ConsultarEstado` *(invocado por `api_front`)*	Retorna última lectura y diagnóstico validando propiedad (R5, R6, R7).
ConsultarHistorialPort	Entrada	`ConsultarHistorial` *(invocado por `api_front`)*	Retorna serie temporal histórica de la planta del usuario (R5, R7).
ListarPlantasPort	Entrada	`ListarPlantas` *(invocado por `api_front`)*	Lista las plantas registradas del usuario autenticado (R7).
RegistrarPlantaPort	Entrada	`RegistrarPlanta` *(invocado por `api_front`)*	Registra planta, vincula sensor y genera token inicial de dispositivo (R7).
EmitirTokenDispositivoPort	Entrada	`EmitirTokenDispositivo` *(invocado por `api_front`)*	Permite al dueño reemitir la credencial JWT de su ESP32 (R7).
AutenticarUsuarioPort	Entrada	`AutenticarUsuario` *(invocado por `api_front`)*	Registro, login, validación de hash y emisión de JWT de sesión (R7).
TablaReferenciaPort	Salida	`CsvTablaReferenciaAdapter` / `BdTablaReferenciaAdapter`	Catálogo botánico. Conmutable por `TABLA_REFERENCIA_DRIVER` (R4).
HistorialLecturasPort	Salida	`BdHistorialLecturasAdapter` (`historial_bd.py`)	Persistencia y consulta histórica de lecturas en SQLite (R5).
UsuariosPort	Salida	`BdUsuariosAdapter` (`usuarios_bd.py`)	Almacenamiento y búsqueda de usuarios en SQLite (R7).
PlantasPort	Salida	`BdPlantasAdapter` (`plantas_bd.py`)	Persistencia de plantas y relación planta-sensor en SQLite (R7).
EmisorTokenPort	Salida	`JwtEmisorTokenAdapter` (`emisor_jwt.py`)	Firma de tokens JWT HS256 con scopes segregados (R7).
HasherContrasenaPort	Salida	`WerkzeugHasherAdapter` (`hasher_werkzeug.py`)	Generación y verificación de hashes criptográficos scrypt (R7).

3. Manejo de Seguridad, Alcances de JWT y Regla de Propiedad

La autenticación y autorización protegen el sistema en el perímetro sin invadir el núcleo del dominio:

- **Segregación de Alcances (User vs Device):** El claim `scope` define dos tipos de token disyuntos. El **token de usuario** (`scope="usuario"`), emitido en `/api/auth/login`, autoriza la consulta y gestión de plantas propias. El **token de dispositivo** (`scope="dispositivo"`), asignado al ESP32, autoriza únicamente el reporte de telemetría en `POST /api/dispositivos/{id}/lecturas`.
- **Puntos de Emisión y Verificación:** La emisión se delega al puerto de salida `EmisorTokenPort` (`JwtEmisorTokenAdapter` en `emisor_jwt.py`, único punto con `PyJWT`). La verificación perimetral ocurre en `seguridad.py` (`JwtVerificadorToken`), donde `@requiere_jwt_usuario` y `@requiere_jwt_dispositivo` rechazan con HTTP 403 Forbidden cualquier intento de cruzar alcances antes de invocar al dominio.
- **Aislamiento del Dominio:** El dominio desconoce JWT y encabezados HTTP. El filtro perimetral extrae la identidad limpia (`usuario_id` o `dispositivo_id`) y la pasa como argumento, permitiendo probar el dominio con dobles en memoria sin claves criptográficas.
- **Regla de Propiedad en el Dominio:** La verificación de pertenencia es una regla de negocio y reside en el dominio:

```
# matera/dominio/casos_uso/propiedad.py
def obtener_planta_del_usuario(plantas: PlantasPort, usuario_id: str, planta_id: str) -> Planta:
    planta = plantas.obtener_planta_por_id(planta_id)
    if planta is None: raise PlantaNoEncontradaError(planta_id)
    if not planta.pertenece_a(usuario_id): raise AccesoNoAutorizadoPlantaError(usuario_id, planta_id)
    return planta
```

Tanto `ConsultarEstado` como `ConsultarHistorial` utilizan esta función. Si un usuario intenta acceder a una planta ajena, el dominio lanza `AccesoNoAutorizadoPlantaError`, y la capa HTTP (`errores_http.py`) responde con HTTP 403 (`ACCESO_DENEGADO`).

4. Justificación de Principios SOLID en el Código

La estructura modular y atómica del proyecto aplica rigurosamente los principios de diseño orientado a objetos:

- **Inversión de Dependencias (DIP):** El núcleo de dominio no depende de bibliotecas externas ni de adaptadores concretos; depende exclusivamente de abstracciones (puertos). Por ejemplo, el caso de uso `RegistrarLectura` recibe interfaces abstractas (`TablaReferenciaPort`, `PlantasPort`, `HistorialLecturasPort`):

```
# matera/dominio/casos_uso/registrar_lectura.py
class RegistrarLectura(RegistrarLecturaPort):
    def __init__(self, tabla_referencia: TablaReferenciaPort, plantas: PlantasPort, historial: HistorialLecturasPort):
        self._tabla_referencia = tabla_referencia
        self._plantas = plantas
        self._historial = historial
```

Ni los casos de uso ni el evaluador botánico importan `sqlite3`, `csv`, `flask` o `jwt`.

- **Abierto/Cerrado (OCP):** El sistema permite incorporar nuevas fuentes o canales mediante la adición de adaptadores, manteniendo el dominio cerrado a modificaciones. Al añadir la telemetría del ESP32 (R2) y el simulador (R3), únicamente construimos `api_dispositivos.py` y `simulador_esp32.py`; el evaluador botánico no requirió ajustes. Igualmente, alternar entre catálogo CSV y SQLite (R4) se resuelve en el Composition Root (`matera/app.py`) sin tocar una sola línea de lógica de negocio.
- **Segregación de Interfaces (ISP):** Refactorizamos los antiguos puertos monolíticos en contratos atómicos y especializados. En lugar de un puerto sobrecargado, existen interfaces pequeñas y enfocadas: `UsuariosPort`, `PlantasPort`, `HistorialLecturasPort`, `TablaReferenciaPort`, `EmisorTokenPort` y `HasherContrasenaPort`. Cada caso de uso recibe únicamente los puertos que consume; por ejemplo, `ListarPlantas` solo recibe `PlantasPort`, permaneciendo completamente ajeno a contraseñas, tokens, historiales o catálogos.
- **Responsabilidad Única (SRP):** Cada clase tiene una única razón de cambio. Cada caso de uso atómico (`ValidarLectura`, `ConsultarEstado`, etc.) orquesta exactamente una operación del negocio. En la infraestructura, cada adaptador SQLite (`usuarios_bd.py`, `plantas_bd.py`, `historial_bd.py`) responde únicamente al esquema relacional de su entidad, y la infraestructura común de conexión reside en `sqlite_bd.py`.

---

5. Verificación de la Prueba de Cambio: Sensor ESP32

Para verificar que la arquitectura permite integrar nuevos canales sin perturbar las reglas de negocio, la incorporación del sensor ESP32 y su simulador se realizó en un commit dedicado:

- **Identificador de Commit:** `8c4b6d8685aeb2bf2784258d349550a6cbf25812`
- **Mensaje del Commit:** `feat(dispositivos): adaptador de dispositivos y simulador ESP32 (Prueba de Cambio)`
- **Archivos creados y modificados:**
  - `matera/adaptadores/entrada/api_dispositivos.py` (nuevo adaptador REST para el sensor)
  - `simulador_esp32.py` (script autónomo de telemetría y pruebas de robustez)
  - `tests/test_api_dispositivos.py` (suite de integración para el nuevo endpoint)
  - `matera/app.py` (registro del blueprint en el punto de composición)

Comprobación de Invarianza del Dominio

Al auditar los cambios introducidos entre dicho commit y su predecesor sobre las carpetas de negocio:

```
git diff 8c4b6d8~1 8c4b6d8 -- matera/dominio/ matera/dominio/puertos/
```

El resultado no reporta ninguna diferencia. Los archivos `modelo.py`, `validador.py`, `excepciones.py`, `entrada.py` y `salida.py` no sufrieron alteraciones.

Esto confirma que el requerimiento R2 se conectó utilizando el puerto preexistente `RegistrarLecturaPort`, demostrando que la separación entre adaptadores y dominio preserva la estabilidad del núcleo cuando se añaden nuevas fuentes de datos.

6. Análisis de Pregunta de Cambio: Migración de Transporte a MQTT

Durante la sustentación se evalúa la capacidad de evolución del sistema ante un cambio tecnológico imprevisto:

> *"Si mañana las lecturas ambientales de la matera ya no llegaran por solicitudes HTTP REST, sino mediante mensajes asíncronos publicados en un broker MQTT (por ejemplo, en el tópico `matera/dispositivos/{id}/telemetria`), ¿qué componentes del repositorio cambiarían y cuáles permanecerían intactos?"*

Componentes Nuevos a Implementar (Adaptador de Entrada)

1. **Nuevo adaptador de suscripción MQTT (`matera/adaptadores/entrada/suscriptor_mqtt.py`):**
   - Utiliza una librería de cliente MQTT como `paho-mqtt`.
   - Se suscribe al patrón de tópicos `matera/dispositivos/+/telemetria`.
   - Al recibir un mensaje en su callback `on_message`, extrae el identificador del dispositivo del tópico, deserializa el payload JSON y valida la integridad de los campos (`humedad`, `luz`, `temperatura`, `marca_tiempo`).
   - Invoca directamente el puerto de entrada:

```
diagnostico = self._puerto_registro.registrar(
    dispositivo_id=dispositivo_id,
    valores=valores_medidos,
    marca_tiempo=marca_tiempo,
)
```

1. **Suite de pruebas del adaptador (`tests/test_suscriptor_mqtt.py`):**
   - Verifica la recepción de mensajes, el manejo de payloads corruptos y la correcta llamada a `RegistrarLecturaPort` mediante dobles de prueba.

Componentes Existentes a Modificar (Únicamente Composición y Entorno)

1. **Punto único de composición (`matera/app.py`):**
   - Se instancia el nuevo adaptador `suscriptor_mqtt` pasándole la misma instancia existente de `registrar_lectura` (que implementa `RegistrarLecturaPort`).
   - Se inicia el bucle de red del cliente MQTT en segundo plano (`client.loop_start()`).
2. **Dependencias del proyecto (`requirements.txt`):**
   - Se añade la dependencia del cliente MQTT (`paho-mqtt`).

Componentes que Permanecen Intactos

- **Núcleo de dominio (`matera/dominio/*`):** `modelo.py`, `validador.py`, `casos_uso/` y `excepciones.py` no requieren ningún cambio. El caso de uso `RegistrarLectura` desconoce por completo si las lecturas provinieron de un paquete TCP, un socket MQTT o una petición HTTP.
- **Puertos de entrada y salida (`matera/dominio/puertos/*`):** `RegistrarLecturaPort` conserva su firma agnóstica (`dispositivo_id: str`, `valores: Mapping`, `marca_tiempo: str`), ya que fue diseñado pensando en el caso de uso y no en el protocolo.
- **Adaptadores de salida (`matera/adaptadores/salida/*`):** La persistencia en SQLite (`historial_bd.py`), la consulta de plantas (`plantas_bd.py`) y el catálogo (`tabla_csv.py` / `tabla_bd.py`) se ejecutan exactamente igual.

---

7. Conclusión y Síntesis de la Arquitectura

La adopción de arquitectura hexagonal con casos de uso atómicos y puertos segregados en esta segunda fase de SmartPlantBase resuelve el desacoplamiento entre las reglas botánicas y la tecnología externa. Al aislar la autenticación y validación de tokens en los adaptadores primarios, atomizar la persistencia SQLite en adaptadores especializados y centralizar la regla de propiedad en el dominio puro, logramos un sistema robusto, testeable al 100% en memoria con dobles y preparado para admitir nuevos protocolos de transporte (como MQTT) sin deuda técnica.